

### 3622. 判断整除性

简单

相关标签

相关企业

提示

Ax

给你一个正整数  $n$ 。请判断  $n$  是否可以被以下两值之和 整除：

- $n$  的数字和（即其各个位数之和）。
- $n$  的数字积（即其各个位数之积）。

如果  $n$  能被该和整除，返回 `true`；否则，返回 `false`。

**示例 1：**

输入：n = 99

输出：true

解释：

C++

智能模式

```
1 class Solution {
2 public:
3     bool checkDivisibility(int n) {
4         int sumhe = 0;
5         int sumji = 1;
6         int num = n;
7         while(num){
8             sumhe += (num % 10);
9             sumji *= (num % 10);
10            num /= 10;
11        }
12        if(n%(sumhe+sumji)==0)return true;
13        return false;
14    }
15};
```

### LCR 013. 二维区域和检索 - 矩阵不可变

中等

相关标签

相关企业

Ax

给定一个二维矩阵 `matrix`，以下类型的多个请求：

- 计算其子矩形范围内元素的总和，该子矩阵的左上角为 `(row1, col1)`，右下角为 `(row2, col2)`。

实现 `NumMatrix` 类：

- `NumMatrix(int[][] matrix)` 给定整数矩阵 `matrix` 进行初始化
- `int sumRegion(int row1, int col1, int row2, int col2)` 返回左上角 `(row1, col1)`、右下角 `(row2, col2)` 的子矩阵的元素总和。

**示例 1：**

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 0 | 1 | 4 | 2 |
| 5 | 6 | 3 | 2 | 1 |
| 1 | 2 | 0 | 1 | 5 |
| 4 | 1 | 0 | 1 | 7 |
| 1 | 0 | 3 | 0 | 5 |

C++

智能模式

```
1 class NumMatrix {
2 public:
3     //前缀和
4     vector<vector<int>>> sums;
5     NumMatrix(vector<vector<int>>& matrix) {
6         //计算行数
7         int m = matrix.size();
8         if(m>0){
9             //求出有多少列
10            int n = matrix[0].size();
11            //开辟空间 m行 n+1列
12            sums.resize(m,vector<int>(n+1));
13            //计算每一行的前缀和
14            for(int i = 0;i<m;i++){
15                for(int j = 0;j<n;j++){
16                    sums[i][j+1] = sums[i][j] + matrix[i][j];
17                }
18            }
19        }
20    }
21
22    int sumRegion(int row1, int col1, int row2, int col2) {
23        int sum = 0;
24        for(int i = row1;i<=row2;i++){
25            sum+=sums[i][col2+1] - sums[i][col1];
26        }
27        return sum;
28    }
29};
```

### LCR 014. 字符串的排列

中等

相关标签

相关企业

Ax

给定两个字符串 `s1` 和 `s2`，写一个函数来判断 `s2` 是否包含 `s1` 的某个变位词。

换句话说，第一个字符串的排列之一是第二个字符串的子集。

**示例 1：**

输入：s1 = "ab" s2 = "eidbaooo"

输出：True

解释：s2 包含 s1 的排列之一 ("ba")。

**示例 2：**

输入：s1= "ab" s2 = "eidbaao"

输出：False

**提示：**

- $1 \leq s1.length, s2.length \leq 10^4$
- `s1` 和 `s2` 仅包含小写字母

**注意：** 本题与主站 567 题相同： <https://leetcode.cn/problems/permutation-in-string/>

面试中遇到这道题？ 1/5

是 否

通过次数 57,327/110.9K 通过率 51.7%

相关标签

相关企业

评论 (202)

C++

智能模式

```
1 class Solution {
2 public:
3     bool checkInclusion(string s1, string s2) {
4         //
5         用哈希表记录 s1 中出现的字符与字符出现次数
6         遍历 s2 除顺序将字符存入队列 并且判断字符是否出现过 如果出现过是否出现次数也是相同
7         如果出现 并且次数相同 那么就匹配上一个字符
8         如果没出现过 去队列队头起了空 如果超出大小 那么就得到即将出队列的元素
9         如果即将出队列的元素 如果没被使用 出队次数不符合了 那 match 也要减少
10        比如 s1 是 abc 字符串是 aaccbb 那么 a 和 b 都是 2 是符合的 所以 match++ 但是如果第一个 a 即将
11        出队列 那么就不符合了 需要 match 减少
12        每一轮去判断 match 是否匹配 如果匹配直接返回即可
13    }
14    int len = s1.size();
15    unordered_map<char,int>mp;
16    for(auto ch:s1){
17        mp[ch]++;
18    }
19    queue<char> q;
20    unordered_map<char,int>have;
21    int match = 0;
22
23    for(auto ch:s2){
24        q.push(ch);
25        have[ch]++;
26
27        // 只有 mp 里面有这个字符，才参与 match 统计！避免插入垃圾 key
28        if(mp.count(ch) && have[ch] == mp[ch]){
29            match++;
30        }
31
32        //窗口滑动长度，弹出队头
33        if(q.size()==len){
34            char out = q.front(); //先拿到要出队的字符
35            //弹出后，如果刚好相等，match 要减
36            if(mp.count(out) && have[out] == mp[out]){
37                match--;
38            }
39            have[out]--; //计数减少
40            q.pop();
41        }
42
43        //全部种类匹配，找到答案
44        if(match == mp.size()){
45            return true;
46        }
47    }
48    return false;
49 }
50};
```